I.1 Reading files

In Linux, almost everything is represented as a file: the keyboard, hard disks, terminals, printers, and even communication devices are exposed through the filesystem interface. Regular files are also represented in the same way. Files can be opened to perform read and/or write operations, and each file has associated attributes such as size, access permissions, ownership, and type.
Like most Unix-like operating systems, Linux defines three standard streams as the primary input/output interfaces for programs: *standard input*, *standard output*, and *standard error*. In Linux, these streams are represented as file descriptors and are identified by both a name and a numeric value.

`stdin`	`0`	Standard input
`stdout`	`1`	Standard output
`stderr`	`2`	Standard error

Tab1. The main I/O devices

When we execute a command from the shell, its output is normally displayed on the *standard output*, while error messages are sent to the *standard error* stream. *Standard input* represents the source from which data is read, typically the keyboard. *Standard output* may correspond to the terminal, a printer, or a file used to store the results of program execution. Likewise, *standard input* and *standard error* can also be redirected to files or other devices instead of the console.

I.1.1 Opening files: `open` and `close`

The `open`[1] function opens the file specified by its first argument according to the mode indicated by the second argument, and returns a *file descriptor*. A file descriptor is a positive integer used by the operating system to identify an open file within a program. Every file opened by a process in the system is associated with a file descriptor.

```
#include <fcntl.h>

int open(const char *pathname, int flags);

                                Returns: the new file descriptor, -1 on error
```

The `openfile.c` program provides an example of how this works.

```
1 #include <stdio.h>
2 #include <fcntl.h>

3 int main (int argc, char *argv[])
4 {
5     int fd;
```

1 See the `open(2)` manual page

```
6     int fd2;
7     int fd3;

8     fd = open("file.txt", O_RDONLY);
9     fd2 = open("file2.txt", O_WRONLY);
10    fd3 = open("file3.txt", O_RDWR);
11    printf("fd = %d", fd);
12    printf("\nfd2 = %d", fd2);
13    printf("\nfd3 = %d", fd3);
14    return 0;
15 }
```

openfile.c

The program opens three files using `open`, each with a different file opening mode, and then prints the file descriptor values returned by the `open` calls.

We create two files and then run the program.

```
[linux@io] touch file.txt file2.txt
[linux@io] ./openfile
fd = 3
fd2 = 4
fd3 = -1
```

The value of the third descriptor is `-1` because the file we attempted to open does not exist. The first descriptor has the value `3` because the descriptors `0`, `1`, and `2` are already reserved for `stdin`, `stdout`, and `stderr`, respectively. For each additional file opened, the operating system typically assigns the next available file descriptor. Therefore, if `file3.txt` existed and were opened successfully, its descriptor would likely be `5`, and so on.

The vluaes the `flags` argument may have, are listed in the following table.

`O_RDONLY`	Read only
`O_WRONLY`	Write only
`O_RDWR`	Read/Write
`O_APPEND`	Append

Tab 2. Opening modes (`open`)

In the previous program, we did not close the files, even though closing files is a fundamental operation when working with them. Although all open file descriptors are automatically closed

when the program terminates, it is good practice to close them explicitly to ensure correctness and avoid wasting system resources.

The `close`[2] function closes the file associated with the file descriptor passed as an argument.

```
#include <unistd.h>

int close(int fd);

                                                Returns: 0 on success, -1 on error
```

The `closefile.c` program shows an example.

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>

4 int main (int argc, char *argv[])
5 {
6     int fd;
7     open("file.txt", O_RDONLY);
8     close(fd);
9     return 0;
10 }
```

closefile.c

After opening the file in read-mode we close it using the `close` function.

We execute the program

```
[linux@io] ./closefile
[linux@io]
```

The program opens and then closes the file.

2 See the `close(2)` manual page

I.1.2 Reading from file: `read`

When we want to read the file contents we can use the `read`[3] function, which reads `count` characters from the `fd` file descriptor and stores them in `buf`.

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count);

                                    Returns: the number of bytes read, -1 on error
```

`read` returns the number of bytes actually read. In some cases, this value may be smaller than the number of bytes requested, for example, when the *end of file* is reached before the requested amount has been read, when reading from a terminal device, a network device, or a pipe, or when the operation is interrupted by a signal.
After a successful call, the *file offset* is automatically increased by the number of bytes read.

The `readfromfile.c` program reads from a file.

```
1  #include <stdio.h>
2  #include <fcntl.h>
3  #include <unistd.h>

4  #define COUNT 128
5  int main (int argc, char *argv[])
6  {
7      int fd;
8      int n;
9      char buffer[COUNT];

10     fd = open("./file.txt", O_RDONLY);
11     if (fd < 0) {
12           printf("Open error - exit..");
13           return 1;
14     }

15     n = read(fd, buffer, COUNT);
16     printf("Bytes read: %d", n);
17     printf("\nContent: %s", buffer);
18     close(fd);
19     return 0;
20 }
```

readfromfile.c

We use a 128-byte buffer to store the data. After the `read` call, we print the number of bytes read along with the contents of the buffer.
We write some data into the `file.txt` file and then run the program.

3 See the `read(2)` manual page

```
[linux@io] echo Linux > file.txt
[linux@io] ./readfromfile
Readed bytes: 6
Content: Linux
```

The buffer size is 128 bytes, but the file contains only 6 bytes. The `read` function reads data from the file up to the specified buffer size (128 bytes). This means it will return up to 128 bytes if available; however, since the file is smaller, only 6 bytes are actually read, and no additional data is returned beyond the end of the file.

I.1.3 Writing to file: `write`

The `write`[4] function is used to write data to a file descriptor. It writes up to `count` bytes from the buffer `buf` to the file descriptor `fd` and returns the number of bytes actually written.

```
#include <unistd.h>

ssize_t write(int fd, const void *buf, size_t count);

                                Returns: the number of written bytes, -1 on error
```

For the *standard input*, *standard output* and *standard error*, the corresponding constants are defined in the `unistd.h` header file. These constants refer to their respective file descriptors.

STDIN_FILENO	0
STDOUT_FILENO	1
STDERR_FILENO	2

Tab 3. Standard descriptors

The `readfile.c` program opens a file, repeatedly reads from it until at least one byte is obtained, prints the number of bytes read along with the data, and then closes the file.

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>

4 #define COUNT 128

5 int main()
6 {
7     int fd;
8     char buffer[COUNT];

9     fd = open("./file.txt", O_RDONLY);
10    if (fd < 0) {
11          printf("Open error - exit..");
12          return 1;
13    }

14    while(read(fd, buffer, COUNT) > 0)
15          write (STDOUT_FILENO, buffer, COUNT);

16    close(fd);
17    return 0;
18 }
```

4 See the `write(2)` manual page

readfile.c

The `while` loop at line 14 calls `read` to retrieve `COUNT` bytes of data from the file referenced by the `fd` descriptor, and the `write` call at line 15 prints the contents of the buffer to `stdout`.

We execute the program

```
[linux@io] ./readfile
Linux
```

Unlike the `readfromfile.c` program, the `readfile.c` program reads from the file until data is available. Each call to `read` retrieves up to 128 bytes from the file. Let's run both programs using a file larger than 128 bytes.

```
[linux@io] cp /etc/passwd file.txt
[linux@io] ./readfromfile
Bytes read: 128
Content: io:1000:1000::/home/io:/usr/bin/zsh
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

[linux@io] ./readfile
io:1000:1000::/home/io:/usr/bin/zsh
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
...output...
```

The `readfromfile.c` program reads 128 bytes from the file, while the `readfile.c` program continues reading the entire file in chunks of 128 bytes (the size of the buffer). The size of `buf` can be greater than `count`, but `write` will still write only `count` bytes. The data type `ssize_t`[5] is used for counting bytes and is defined in the `sys/types.h` and `unistd.h` header files.

5 See the `system_data_types(7)` manual page

I.1.4 The standard library functions: `fopen`, `fclose` and `fcloseall`

The `open`, `read`, `close`, and `write` functions are *system calls*. The *standard library* provides a set of file I/O functions that allow files to be handled as *streams*. These functions are not Linux system calls; they belong to the *C standard library* and are defined in the `stdio.h` header file.

The `fopen`[6] function is used to open a file and associate it with a *stream*. It takes as arguments the file name and the opening mode, and returns a pointer to the corresponding *file stream*. The `fclose`[7] function closes the file associated with the stream passed as its argument.

```
#include <stdio.h>

FILE *fopen(const char *pathname, const char *mode);

                                    Return: a pointer to the file, NULL on error

int fclose(FILE *stream);

                                                Returns: 0 on success, EOF on error
```

When working with multiple open files, it may be useful to close all streams at once. The `fcloseall`[8] function closes all streams opened by the process.

```
#include <stdio.h>

int fcloseall(void);

                                                Returns: 0 on success, EOF on error
```

6 See the `fopen(3)` manual page
7 See the `fclose(3)` manual page
8 See the `fcloseall(3)` manual page

I.1.5 Reading one character at a time: `fgetc` and `fputc`

The `fgetc`[9] and `fputc`[10] functions are used to read and write a single character, respectively. The first reads a character from the specified input stream and returns it. The second writes a character to the specified output stream and returns the written character.

```
#include <stdio.h>

int fgetc(FILE *stream);
                                        Returns: the read character, EOF on error

int fputc(int c, FILE *stream);
                                      Return: the written character, EOF on error
```

The `readfilef.c` program reads the contents of the file and prints them to the screen.

```
1 #include <stdio.h>

2 int main()
3 {
4     FILE *file;
5     char c;

6     file = fopen("./file.txt", "r");
7     while (c!=EOF) {
8           c=fgetc(file);
9           if(c!=EOF)
10                fputc(c, stdout);
11    }
12    fclose(file);
13    return 0;
14 }
```

readfilef.c Read one character at a time

The `while` loop continues iterating until `c` becomes `EOF` (End of File). In each iteration, it reads one byte from the file using `fgetc`, and if the byte read is valid, it prints it to `stdout`.

file.txt

```
Linux Programming
1234567890
```

The program is executed

```
[linux@io] ./readfilef
Linux Programming
```

9 See the `fgetc(3)` manual page
10 See the `puts(3)` manual page

```
1234567890
```

The `FILE` data type is an object used to represent streams, and it is defined in the `stdio.h` header file.
The standard library also defines constants in `stdio.h` for standard input, standard output, standard error, and end-of-file handling.

`stdin`	`0`
`stdout`	`1`
`stderr`	`2`
`EOF`	`-1`

Tab 4. The standard streams

We present the same program using a more concise syntax.

```
1 #include <stdio.h>

2 int main()
3 {
4     char c;
5     FILE *fp;
6     fp = fopen("./file.txt", "r");
7     while ((c=fgetc(fp)) != EOF)
8           fputc(c, stdout);

9     fclose(fp);
10    return 0;
11 }
```

readfilefs.c

The program is executed

```
[linux@io] ./readfilefs
Linux Programming
1234567890
```

The `fopen` function accepts the following flags:

`r`	Read
`r+`	Read and Write
`w`	Write. If the file exists its contents are truncated to zero

w+	Read and Write. If the file does not exist it is created
a	Writes at the end of the file (appends to existing content)
a+	Reading and queuing. If the file does not exist it is created

Tab 5. File opening modes of `fopen`

In addition to the standard flags `fopen` also accepts the following

c	The file opens with deletion disabled
e	The descriptor will be closed if one of the `exec`[11] functions is used (corresponds to the `FD_CLOEXEC`)
m	The file is opened and accessed via `mmap`[12]
x	Open the file exclusively

Tab 6. `fopen` flags

The correspondence between the `fopen` and `open` flags is shown in the following table.

fopen	**open**
r	O_RDONLY
w	O_WRONLY | O_CREAT | O_TRUNC
a	O_WRONLY | O_CREAT | O_APPEND
r+	O_RDWR
w+	O_RDWR | O_CREAT | O_TRUNC
a+	O_RDWR | O_CREAT | O_APPEND

Tab 7. File opening methods (`open` and `fopen`)

The `stdin2stdout.c` program copies data from standard input to the standard output, one character at a time.

```
1 #include <stdio.h>

2 int main()
3 {
4     char c;

5     while(c=fgetc(stdin))
6           fputc(c, stdout);
7 }
```

11 See the `exec(3)` manual page
12 See the `mmap(2)` manual page

stdin2stdout.c

The program reads one character at a time from standard input (`stdin`) and prints it to standard output (`stdout`), effectively echoing the characters typed by the user onto the screen.

```
[linux@io] ./Stdin2Stdout
hello
hello
Linux
Linux
```

I.1.6 Reading one line at a time: `fgets` and `fputs`

The `fgets`[13] function reads a line from the file passed as an argument and stores it in a buffer. It takes as parameters the buffer where the line will be stored, the maximum number of bytes to read, and the file stream from which to read, and it returns the resulting string.
The `fputs`[14] function writes a line to the file passed as an argument. It takes as parameters the buffer containing the data to be written and the output file stream.

```
#include <stdio.h>

char *fgets(char *s, int size, FILE *stream);

                                        Return: the buffer read, NULL on error

int fputs(const char *s, FILE *stream);

                                    Return: a non negative integer, EOF on error
```

The `readfileline.c` program uses `fgets` to read the file one line at a time.

```
1 #include <stdio.h>

2 int main()
3 {
4     FILE *file;
5     char buffer[1024];
6     int count = 1024;

7     if ((file = fopen("./file.txt", "r")) == NULL) {
8          printf ("Error opening file");
9          return 1;
10    }
11    while (fgets (buffer, count, file))
12         fputs (buffer, stdout);

13    fclose (file);
14    return 0;
15 }
```

---

readfileline.c

This version of the program uses `fopen` to open the file, `fgets` to read from the stream, and `fputs` to output the read data to the screen.

```
[linux@io] ./readfileline
Hello world
```

13 See the `fgetc(3)` manual page
14 See the `puts(3)` manual page

```
1234567890
```

The program reads the file one line at a time and prints it to the screen. The `fgets` function attempts to read up to `count - 1` bytes from the stream; if it encounters an end-of-file (`EOF`) or a newline character, it stops reading.

The `stdin2stdoutline.c` program copies standard input to standard output, line by line.

```
1 #include <stdio.h>

2 int main()
3 {
4     char buf[256];

5     while(fgets(buf, 256, stdin) != NULL)
6           fputs(buf, stdout);
7 }
```

---

stdin2stdoutline.c

The `fgets` call attempts to read up to 256 bytes from standard input until it encounters an `EOF` or a newline character, and stores the data in the buffer `buf`. `fputs` writes the contents of `buf` to standard output.

```
[linux@io] ./stdin2stdoutline
hello Linux
hello Linux
```

The program copies each input line to standard output.